

SKILL - WEEK : 10

SESSION 19 :

Task -1 :

To Register Handlers, Configure Signal Actions, Handle Interrupts, Manage Signal Context, Verify Stability, Test Signal Events

CODE:

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <signal.h>

void handle_signal(int sig, siginfo_t *info, void *context)
{
    printf("\nSignal %d received.\n", sig);
    printf("Signal handled successfully.\n");
    printf("Process PID: %d\n", getpid());
}

int main()
{
    struct sigaction action;

    action.sa_sigaction = handle_signal;
    sigemptyset(&action.sa_mask);
    action.sa_flags = SA_SIGINFO;

    sigaction(SIGINT, &action, NULL);

    printf("Signal handler registered.\n");
    printf("Process PID: %d\n", getpid());
    printf("Press Ctrl+C to generate SIGINT.\n");

    for (int i = 1; i <= 10; i++)
    {
        printf("Program running... %d\n", i);
        sleep(2);
    }

    printf("Program completed successfully.\n");

    return 0;
}
```

OUTPUT:

```
ananya@Ananya:~$ cd /mnt/d/OSSP_Programs/Skill/Week10
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week10$ nano signal_action.c
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week10$ gcc signal_action.c -o signal_action
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week10$ ./signal_action
Signal handler registered.
Process PID: 1170
Press Ctrl+C to generate SIGINT.
Program running... 1
Program running... 2
Program running... 3
Program running... 4
Program running... 5
Program running... 6
Program running... 7
Program running... 8
Program running... 9
Program running... 10
^C
Signal 2 received.
Signal handled successfully.
Process PID: 1170
Program completed successfully.
```

OBSERVATION:

The program successfully demonstrated signal registration, signal handling, and interrupt management using `sigaction()`. A custom handler was registered for `SIGINT`, and the program continued executing normally until `Ctrl+C` was pressed. The generated `SIGINT` was successfully received and handled by the custom signal handler, which displayed the signal number and process ID. Instead of terminating immediately, the program completed its execution successfully, demonstrating controlled interrupt handling and stable signal behavior.

Task - 2:

To Capture SIGINT, Forward Signals, Protect Shell Process, Terminate Foreground Jobs, Update Job States, Test Interrupt Handling

CODE:

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

pid_t child_pid;

void handle_sigint(int sig)
{
    printf("\nShell: SIGINT received.\n");

    if (child_pid > 0)
    {
        printf("Shell: Forwarding SIGINT to foreground job.\n");
        kill(child_pid, SIGINT);
    }
}

int main()
{
    int status;

    signal(SIGINT, handle_sigint);

    child_pid = fork();

    if (child_pid < 0)
    {
        perror("fork");
        return 1;
    }

    if (child_pid == 0)
    {
        signal(SIGINT, SIG_DFL);

        printf("Foreground job started. PID = %d\n", getpid());

        while (1)
        {
            printf("Foreground job running...\n");
            sleep(2);
        }
    }
}
```

```
printf("Shell: Foreground job created. PID = %d\n", child_pid);
printf("Press Ctrl+C to interrupt the foreground job.\n\n");

waitpid(child_pid, &status, 0);

if (WIFSIGNALED(status))
{
    printf("\nShell: Foreground job terminated by signal %d.\n",
        WTERMSIG(status));
    printf("Shell: Job state updated to Terminated.\n");
}

printf("Shell: Still running normally.\n");

return 0;
}
```

OUTPUT:

```
ananya@Ananya:~$ cd /mnt/d/OSSP_Programs/Skill/Week10
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week10$ nano forward_signal.c
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week10$ gcc forward_signal.c -o forward_signal
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week10$ ./forward_signal
Shell: Foreground job created. PID = 1269
Press Ctrl+C to interrupt the foreground job.

Foreground job started. PID = 1269
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
Foreground job running...
^C
Shell: SIGINT received.
Shell: Forwarding SIGINT to foreground job.

Shell: Foreground job terminated by signal 2.
Shell: Job state updated to Terminated.
Shell: Still running normally.
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week10$ █
```

OBSERVATION:

The program successfully demonstrated SIGINT capture, signal forwarding, foreground job termination, and job-state management. The parent process acted as a shell and created a foreground child process. When Ctrl+C was pressed, the shell successfully captured SIGINT and forwarded the signal to the foreground job. The child process was terminated by signal 2, and the parent detected the termination using `waitpid()`. The job status was then updated to Terminated, while the shell continued running normally. Thus, interrupt handling and protection of the shell process were successfully demonstrated.

SESSION 20 :

Task -1 :

To Capture SIGTSTP, Suspend Processes, Update Job Table, Preserve State, Resume Later, Test Suspension Workflow

CODE:

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

struct Job
{
    int id;
    pid_t pid;
    char status[20];
};

int main()
{
    struct Job job;
    pid_t pid;
    int status;

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child: Job started. PID = %d\n", getpid());

        while (1)
        {
            printf("Child: Job running...\n");
            sleep(2);
        }
    }

    job.id = 1;
    job.pid = pid;
    sprintf(job.status, "Running");

    printf("Parent: Job created.\n");
    printf("Job ID: %d\n", job.id);
    printf("PID: %d\n", job.pid);
    printf("Status: %s\n", job.status);

    sleep(3);
}
```

```

printf("\nParent: Sending SIGTSTP to stop job...\n");
kill(pid, SIGTSTP);
waitpid(pid, &status, WUNTRACED);
if (WIFSTOPPED(status))
{
    sprintf(job.status, "Stopped");
    printf("Job status: %s\n", job.status);
}

printf("\nParent: Job state preserved.\n");
sleep(2);

printf("Parent: Sending SIGCONT to resume job...\n");
kill(pid, SIGCONT);

sprintf(job.status, "Running");
printf("Job status: %s\n", job.status);

sleep(5);

printf("\nParent: Terminating job...\n");
kill(pid, SIGTERM);

waitpid(pid, &status, 0);

sprintf(job.status, "Completed");
printf("Job status: %s\n", job.status);
printf("Suspension and resume workflow completed.\n");

return 0;
}

```

OUTPUT:

```

ananya@Ananya: /mnt/d/OSSP_Programs/Skill/Week10$ nano suspend_job.c
ananya@Ananya: /mnt/d/OSSP_Programs/Skill/Week10$ gcc suspend_job.c -o suspend_job
ananya@Ananya: /mnt/d/OSSP_Programs/Skill/Week10$ ./suspend_job
Parent: Job created.
Job ID: 1
PID: 1360
Status: Running
Child: Job started. PID = 1360
Child: Job running...
Child: Job running...

Parent: Sending SIGTSTP to stop job...
Job status: Stopped

Parent: Job state preserved.
Parent: Sending SIGCONT to resume job...
Job status: Running
Child: Job running...
Child: Job running...
Child: Job running...

Parent: Terminating job...
Job status: Completed
Suspension and resume workflow completed.
ananya@Ananya: /mnt/d/OSSP_Programs/Skill/Week10$

```

OBSERVATION:

The program successfully demonstrated the suspension and resumption of a background job using signals. A child process was created and maintained in a job table with its Job ID, PID, and current status. The job initially ran normally and was then suspended using SIGTSTP. The stopped state was successfully detected using waitpid() with WUNTRACED, and the job state was preserved. The process was later resumed using SIGCONT, and its status was updated from Stopped to Running. Continued execution of the child confirmed successful recovery. Finally, the job was terminated and its status was updated to Completed, successfully demonstrating the complete suspension and resume workflow.

Task - 2 :

To Create Process Groups, Assign Group IDs, Transfer Terminal Control, Restore Ownership, Coordinate Signals, Validate Behavior

CODE:

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;
    int status;

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid == 0)
    {
        setpgid(0, 0);

        printf("Child: PID = %d\n", getpid());
        printf("Child: Process Group ID = %d\n", getpgrp());

        signal(SIGINT, SIG_DFL);

        while (1)
        {
            printf("Child: Running...\n");
            sleep(2);
        }

        setpgid(pid, pid);

        printf("Parent: Child PID = %d\n", pid);
        printf("Parent: Child Group ID = %d\n", getpgid(pid));

        sleep(2);

        signal(SIGTTOU, SIG_IGN);

        printf("Parent: Transferring terminal control...\n");

        tcsetpgrp(STDIN_FILENO, getpgid(pid));

        printf("Parent: Sending SIGINT to process group...\n");
        kill(-getpgid(pid), SIGINT);

        waitpid(pid, &status, 0);

        printf("Parent: Child process group terminated.\n");

        printf("Parent: Restoring terminal control...\n");

        tcsetpgrp(STDIN_FILENO, getpgrp());

        printf("Parent: Terminal control restored successfully.\n");

        return 0;
    }
}
```

OUTPUT:

```
ananya@Ananya:~$ cd /mnt/d/OSSP_Programs/Skill/Week10
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week10$ nano process_group.c
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week10$ gcc process_group.c -o process_group
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week10$ ./process_group
Parent: Child PID = 1471
Parent: Child Group ID = 1471
Child: PID = 1471
Child: Process Group ID = 1471
Child: Running...
Child: Running...
Parent: Transferring terminal control...
Parent: Sending SIGINT to process group...
Parent: Child process group terminated.
Parent: Restoring terminal control...
Parent: Terminal control restored successfully.
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week10$
```

OBSERVATION:

The program successfully demonstrated process-group creation, process-group identification, terminal control transfer, signal coordination, and restoration of terminal ownership. A child process was created and assigned to its own process group using `setpgid()`. The process group ID was verified using `getpgrp()` and `getpgid()`. Terminal control was then transferred to the child process group using `tcsetpgrp()`, and `SIGINT` was sent to the process group to terminate the foreground job. After the child process group terminated, the parent restored terminal control to itself. Thus, process-group management and terminal ownership were successfully validated.